

FCN

Fully Convolutional Networks for Semantic Segmentation(이하 FCN)은 이미 제목에 드러난 것처럼 Semantic Segmentation 문제를 위해 제안된 딥러닝 모델이다.



입력 이미지에 대해 픽셀단위로 배경 및 각 개체의 클래스를 분할하는 동시에 예측한다.

FCN은 Semantic Segmentation 모델을 위해 기존에 이미지 분류에서 우수한 성능을 보인 CNN 기반 모델(AlexNet, VGG16, GoogLeNet)을 목적에 맞춰 변형시킨 것이다.

이러한 [Image classification model] to [Semantic segmentation model]은 크게 다음의 세 과정으로 표현할 수 있다:

- ***Convolutionalization***
- ***Deconvolution (Upsampling)***
- ***Skip architecture***

이번 리뷰의 핵심은 각 과정에 대한 방법과 의미를 이해하는 것이다.

Convolutionalization

Convolutionalization(컨볼루션화)이라는 표현의 의미를 이해하기 위해서는 기존의 이미지 분류 모델들을 먼저 살펴볼 필요가 있다.



Image classification 모델들은 기본적으로 내부 구조와 관계없이 모델의 근본적인 목표를 위해 출력층이 Fully-connected(이하 fc) layer로 구성되어 있다.

이러한 구성은 네트워크의 입력층에서 중간부분까지 ConvNet을 이용하여 영상의 특징들을 추출하고 해당 특징들을 출력층 부분에서 fc를 이용해 이미지를 분류하기 위함이다.

그런데, Semantic Segmentation 관점에서는 **fc layer가 갖는 한계점**이 있다.

1. 이미지의 위치 정보가 사라진다.



fully-connected layer 연산 이후 Receptive field 개념이 사라진다.

2. 입력 이미지 크기가 고정된다.



Dense layer에 가중치 개수가 고정되어 있기 때문에 바로 앞 레이어의 Feature Map의 크기도 고정되며, 연쇄적으로 각 레이어의 Feature Map 크기와 Input Image 크기 역시 고정된다.

Segmentation의 목적은 **원본 이미지의 각 픽셀에 대해 클래스를 구분하고 인스턴스 및 배경을 분할하는 것**으로 위치 정보가 매우 중요하다.



Segmentation은 입력 이미지의 위치 정보가 유지되어야 한다.

이러한 fc-layer의 한계를 보완하기 위해 모든 fc-layer를 Conv-layer로 대체하는 방법을 택하였다.



Fully-connected layer는 입력의 모든 영역을 receptive field로 보는 필터의 Conv layer로 생각할 수 있다.

VGG16을 예로 살펴보자. 다음과 같이 출력층 부분의 마지막 3 fc-layers를 모두 Conv-layers로 변경한다.



Dense Layer에서 Conv Layer로 변환하는 방식은 다음과 같다.



첫 번째 fully-connected layer를 $(7 \times 7 \times 512)$ 4096filter conv로 변경하면 가중치의 수가 유지된다.



마지막 fc-layer의 경우 채널 차원을 클래스 수에 맞춘 1x1 conv로 변환한다.

VGG16에서 다섯 번째 max-pooling(size: 2×2, stride: 2) 연산 후 Feature map의 크기는 7x7이 된다. (입력 이미지의 크기가 224x224 인 경우)



Convolutionalization을 통해 출력 Feature map은 원본 이미지의 위치 정보를 내포할 수 있게 되었다.

그러나 Semantic segmentation의 최종 목적인 픽셀 단위 예측과 비교했을 때, FCN의 출력 Feature map은 너무 **coarse(거친, 알맹이가 큰)** 하다.

따라서, Coarse map을 원본 이미지 크기에 가까운 Dense map으로 변환해줄 필요가 있다. 적어도 $\text{input image size} * 1/32$ 보다는 해상도가 높을 필요가 있다.

Deconvolution

Coarse map에서 Dense map을 얻는 몇 가지 방법이 있다:

- **Interpolation**
- **Deconvolution**
- Unpooling
- Shift and stitch

물론 Pooling을 사용하지 않거나, Pooling의 stride를 줄임으로써 Feature map의 크기가 작아지는 것을 처음부터 피할 수도 있다.

그러나, 이 경우 필터가 더 세밀한 부분을 볼 수는 있지만 Receptive Field가 줄어들어 이미지의 컨텍스트를 놓치게 된다.

또한, **Pooling의 중요한 역할 중 하나**는 특징맵의 크기를 줄임으로써 학습 파라미터의 수를 감소시키는 것인데, 이러한 과정이 사라지면 파라미터의 수가 급격히 증가하고 이로 인해 더 많은 학습시간을 요구하게 된다.

따라서, Coarse Feature map을 Dense map으로 **Upsampling**하는 방법을 고려해야 한다.

Bilinear Interpolation

10×10 이미지를 320×320 이미지로 확대하려면 어떻게 해야할까?

대표적인 방법이 바로 Bilinear interpolation이다. Bilinear interpolation을 이해하기 위해서는 Linear interpolation을 우선적으로 이해할 필요가 있다.

다음의 두 값을 예측해 보자.



위의 값은 5, 아래 값은 7 임을 어렵지 않게 예측할 수 있다. 이 처럼 두 지점 사이의 값을 추정할 때 직관적으로 사용하는 방법이 Linear interpolation이다.

위의 추정 방식은 다음과 같이 식으로 표현할 수 있다.



식을 앞의 예제에 대입하면 다음과 같이 추정값을 쉽게 구할 수 있다.



Bilinear interpolation은 이러한 1차원의 Linear interpolation을 2차원으로 확장한 것이다.



네 지점 A, B, C, D 사이의 임의의 점 X를 추정할 수 있다.

이제 우리는 다음과 같은 Feature map의 빈 영역을 추정할 수 있다.



Backwards convolution

Dense prediction을 위한 Upsampling 방법에는 Bilinear interpolation처럼 정해진 방법만 있는 것은 아니다. 즉, Up-sampling도 학습이 가능하다.

Stride가 2이상인 Convolution 연산의 경우 입력 이미지에 대해 크기가 줄어든 특징맵을 출력한다. 이것은 Down-sampling에 해당한다.



Convolution 연산을 반대로 할 경우 자연스럽게 Up-sampling 효과를 볼 수 있다. 또한, 이 때 사용하는 Filter의 가중치 값은 학습 파라미터에 해당한다.



FCNs에서는 Bilinear interpolation과 Backwards convolution 두 가지 방법을 사용하여 Coarse Feature map으로부터 Dense prediction을 구했다.

초기 Segmentation을 위한 모델은 다음과 같이 VGG모델을 컨볼루션화한 구조에 Bilinear interpolation 작업을 더함으로써 얻을 수 있다.



마지막 레이어에 Up-sampling 연산을 추가해 Dense prediction을 처리함

이 처럼, Interpolation을 통해 coarse map에서 dense map을 도출할 수 있었지만 근본적으로 feature map의 크기가 너무 작기 때문에 예측된 dense map의 정보는 여전히 거칠 수 밖에 없다.



Segmentation이 정교하지 못하다.

무려 Stride가 32인 Up-sampling 과정에서 얼마나 많은 정보가 손실될 지를 생각해보면 당연한 결과이다.

보다 정교한 Segmentation을 위해서 추가적인 작업이 필요한 이유다.

Skip Architecture

본 논문에서는 정확하고 상세한 구분(Segmentation)을 얻기 위해

Deep & Coarse(추상적인) 레이어의 의미적(Semantic) 정보와 **Shallow & fine** 층의 외관적(appearance) 정보를 결합한 **Skip architecture**를 정의한다.



이러한 직관은 *Visualizing and Understanding Convolutional Networks* 연구를 통해서도 엿볼 수 있다.





시각화 모델을 통해 입력 이미지에 대해 얇은 층에서는 주로 직선 및 곡선, 색상 등의 낮은 수준의 특징에 활성화되고, 깊은 층에서는 보다 복잡하고 포괄적인 개체 정보에 활성화된다는 것을 확인할 수 있다.

또한 얇은 층에선 local feature를 깊은 층에선 global feature를 감지한다고 볼 수 있다.

FCNs 연구팀은 이러한 직관을 기반으로 앞에서 구한 Dense map에 얇은 층의 정보를 결합하는 방식으로 Segmentation의 품질을 개선하였다.



FCN-16s



FCN-8s

각 Pooling에 Prediction을 위해 추가된 Conv layer의 필터는 0으로, Trainable Backwards convolution은 Bilinear interpolation으로 초기화한 후 학습을 진행하였다. 이러한 Skip Architecture를 통해 다음과 같이 개선된 Segmentation 결과를 얻을 수 있다.



실제 성능 지표에서도 FCN-32s > FCN-16s > FCN-8s 순으로 결과가 나아지는 것을 확인할 수 있다.





정리

FCNs은 기존의 딥러닝 기반 이미지 분류를 위해 학습이 완료된 모델의 구조를 Semantic Segmentation 목적에 맞게 수정하여 Transfer learning 하였다.

Convolutionalized 모델을 통해 예측된 Coarse map을 원본 이미지 사이즈와 같이 세밀 (Dense)하게 만들기 위해 Up-sampling을 수행하였다.

또한 Deep Neural Network에서 얇은 층의 Local 정보와 깊은 층의 Semantic 정보를 결합하는 Skip architecture를 통해 보다 정교한 Segmantation 결과를 얻을 수 있었다.